Lecture plan · for the lecturer

Prompt injection and LLM apps

AI security · University · Lecture 4

Time	Materials
60 min	Projector or board; The lecture slides and the seminar sheet from the kit; Phones or laptops with Darsi, for the lab

Learning goals

- Explain prompt injection.
- Design defence in depth.
- Run red teaming safely.

Activity: Red team in scope

1. Warm-up (5 min): ماذا سيفعل؟
2. Scope (10 min): Read the scope and rules of engagement together.
3. Attack (20 min): Teams attempt direct and indirect injection and log each try.
4. Fix (20 min): Each team proposes one design change and re-tests.
5. Wrap-up (5 min): Students write the muddiest point of the lecture and one question; read them before next week.

Discussion

- Which attack was hardest to block?
- Did the system prompt help?
- What should need approval?

Exit ticket

- Name one tool you would never give an LLM app without approval.

Try it in your next lecture
Run a twenty-minute red-team round on a sandboxed course assistant under a written scope; teams log each attempt and propose one design change that would have blocked their best attack.

Seminar sheet · for students

Prompt injection and LLM apps

AI security · University · Lecture 4

Name _____ Class _____ Date _____

1 **Which design best limits damage if the assistant is injected?**

◯ A longer system prompt ◯ A phrase blocklist

◯ Least privilege and approval before any send

2 **Score the red team.**

Before defences, 12 of 40 injection attempts succeed: _____ %. After least privilege and approval, 2 of 40 succeed: _____ %.

Word bank: 30 · 12 · 2 · 5

3 **Which reduce the impact of prompt injection? Choose all that apply.**

☐ Minimal tools and scopes ☐ Human approval for sending or deleting

☐ Treating model output as untrusted ☐ A longer, stricter system prompt

4 **A well-written system prompt reliably prevents indirect prompt injection.**

◯ True ◯ False

5 **Match each control to the risk it limits.**

Least privilege	• • • •	Undetected misuse
Human approval	• • • •	Injected code or links reaching other systems
Output treated as untrusted	• • • •	Irreversible actions such as sending
Logging	• • • •	An injected model calling powerful tools

✎ **If your students red-teamed a live university system without permission, what rule would they have broken, and is it written down?**